DTS103TC
Data Structure and Algorithms

Lab 6: Dynamic Programming

Dr. Qi Chen, Dr. Di Zhang, Dr. Md Maruf Hasan,
Dr. Xuechen Liu, Dr. Guangyu Ren and Dr. Bin Chen
School of AI and Advanced Computing

Learning outcome

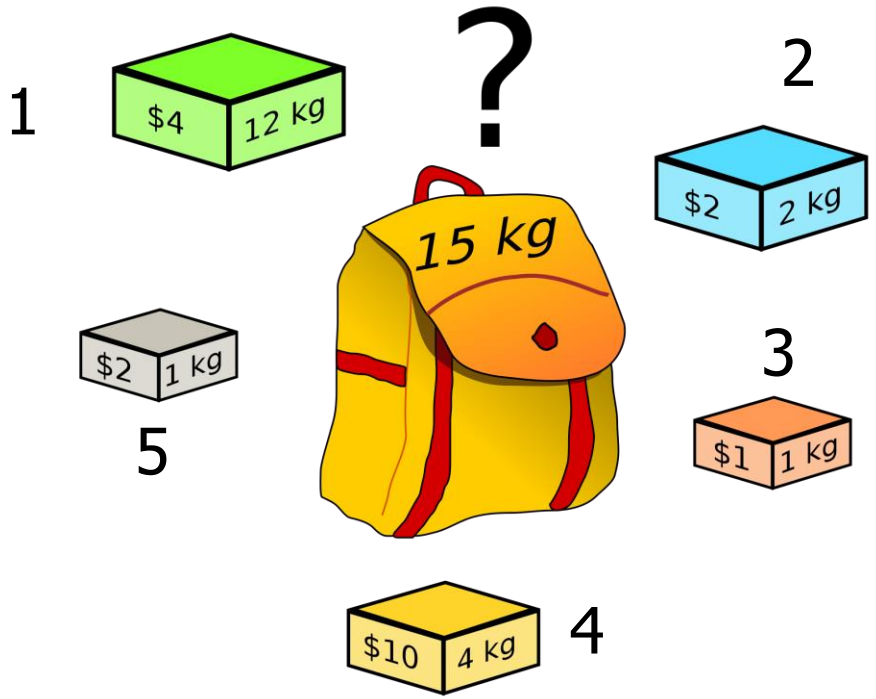
- Able to implement dynamic programming with Python
  - 0-1 knapsack problem
  - longest common subsequence problem
  - Longest Increasing Subsequence
  - Climbing Stairs

How to apply Dynamic Programming?

- Step 1: Identify sub-problems and optimal substructure.
- Step 2: Find a recursive formulation
- Step 3: Use dynamic programming (typically in a bottom-up fashion) to compute the value of an optimal solution
- Step 4: If needed, keep track of some additional info to get the optimal solution

The 0-1 Knapsack Problem

- Given: A set of n items, with each item i having
  - w_i – a positive weight
  - v_i – a positive benefit value
- Goal: Choose items with maximum total value but with weight at most W.



W = 15

Item (i)	1	2	3	4	5
Weight (w)	12	2	1	4	1
Value (v)	4	2	1	10	2

Knapsack problem

Item (i)	1	2	3	4	5
Weight (w)	12	2	1	4	1
Value (v)	4	2	1	10	2

- Fill up the table.

$$f[i, w] = \begin{cases} 0 & \text{if } i=0 \text{ or } w=0 \\ f[i-1, w] & \text{else if } w_i > w \\ \max(f[i-1, w], v_i + f[i-1, w- w_i]) & \text{otherwise} \end{cases}$$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S_0 ={}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
S_1 ={1}	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
S_2 ={1,2}	0	0	2	2	2	2	2	2	2	2	2	2	4	4	6	6
S_3 ={1,2,3}	0	1	2	3	3	3	3	3	3	3	3	3	4	5	6	7
S_4 ={1,2,3,4}	0	1	2	3	10	11	12	13	13	13	13	13	13	13	13	13
S_5 ={1,2,3,4,5}	0	2	3	4	10	12	13	14	15	15	15	15	15	15	15	15

Dynamic Programming Implementation

```
def knapsack(W, weight, value, n):
    f = [[None]*(W+1) for x in range(n + 1)]

    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                f[i][w] = 0
            elif weight[i-1] <= w:
                f[i][w] = max(value[i-1]
                        + f[i-1][w-weight[i-1]],
                            f[i-1][w])
            else:
                f[i][w] = f[i-1][w]

    return f[n][W]
```

```
weight = [12, 2, 1, 4, 1]
value = [4, 2, 1, 10, 2]
W = 15
n = len(weight)
print(knapsack(W, weight, value, n))
```

```
15
```

Backtrack to find the items

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S_0 ={}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
S_1 ={1}	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
S_2 ={1,2}	0	0	2	2	2	2	2	2	2	2	2	2	4	4	6	6
S_3 ={1,2,3}	0	1	2	3	3	3	3	3	3	3	3	3	4	5	6	7
S_4 ={1,2,3,4}	0	1	2	3	10	11	12	13	13	13	13	13	13	13	13	13
S_5 ={1,2,3,4,5}	0	2	3	4	10	12	13	14	15	15	15	15	15	15	15	15

Item (i)	1	2	3	4	5
Weight (w)	12	2	1	4	1
Value (v)	4	2	1	10	2

Item: {2,3,4,5} Max value: 15

```
included_items = []
i, j = n, W
while i > 0 and j > 0:
    if f[i][j] != f[i-1][j]:
        included_items.append(i)
        j -= weight[i-1]
    i -= 1
included_items.reverse()
return f[n][W], included_items
```

Longest Common Subsequence

Longest Common Subsequence (LCS)

- How similar are these two DNA sequences?

1) AGCCCTAAGGGCTACCTAGCTT
2) GACAGCCTACAAGCGTTAGCTTG

- Pretty similar, has a long common subsequence:
  1) AGCCCTAAGGGCTACCTAGCTT
  2) GACAGCCTACAAGCGTTAGCTTG

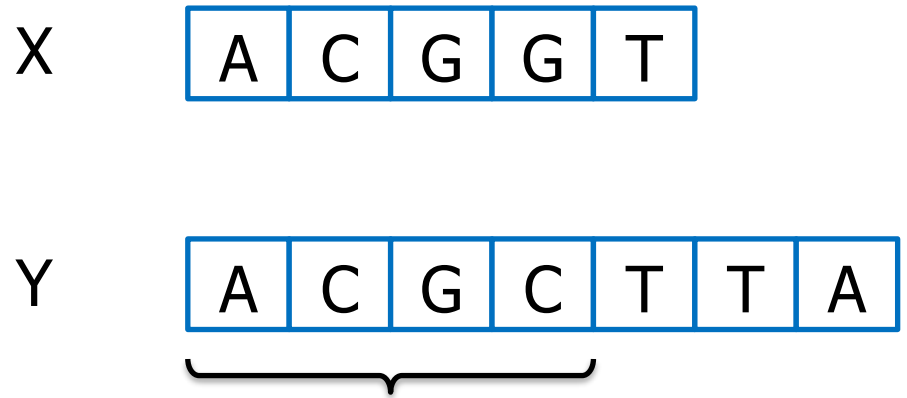
AGCCTAAGCTTAGCTT

Longest Common Subsequence (LCS)

- Subsequence:
  - BDFH is a subsequence of ABCDEFGH
- If X and Y are sequences, a common subsequence is a sequence which is a subsequence of both.
  - BDFH is a common subsequence of ABCDEFGH and of ABDFGHI
- A longest common subsequence…
  - is a common subsequence that is longest.
  - The longest common subsequence of ABCDEFGH and ABDFGHI is ABDFGH.
- Has applications to DNA similarity testing.

Sub-problems

- Prefixes:



Notation: Denote this prefix ACGC by Y_4

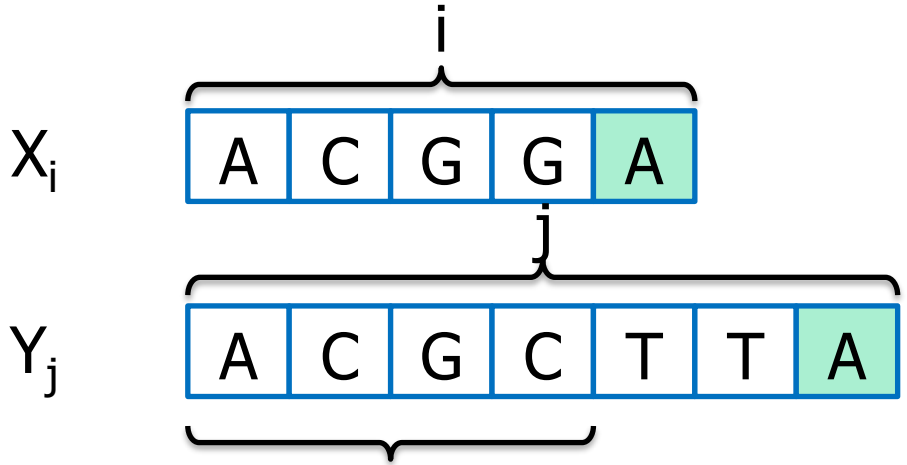
- Our sub-problems will be finding LCS's of prefixes to X and Y.
- Let f[i,j] = length_of_LCS(X_i, Y_j)
- Finding LCS's prefixes of X and Y.

Sub-problems

Our sub-problems will be finding LCS's of prefixes to X and Y.

Let f[i,j] = length_of_LCS(X_i, Y_j)

- *Case 1*: X[i] = Y[j]



Notation: Denote this prefix *ACGC* by Y_4

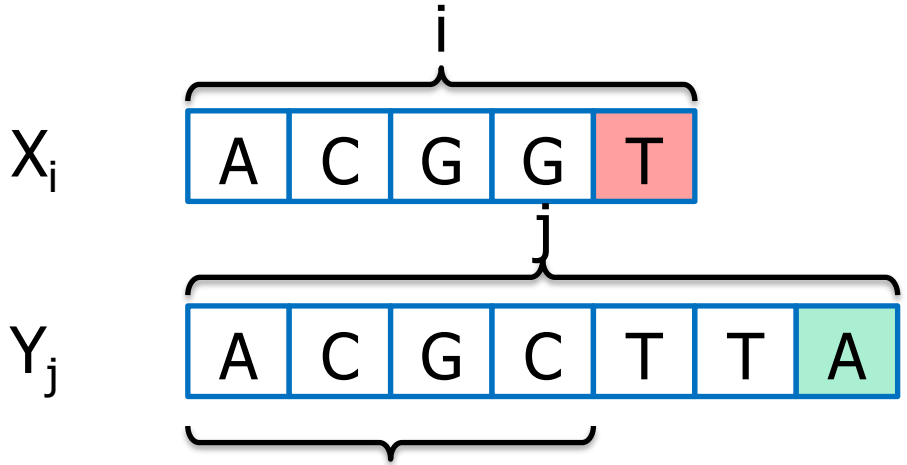
- Then, f[i, j] = 1 + f[i-1,j-1].
  - Because LCS(X_i, Y_j) = LCS(X_{i-1}, Y_{j-1}) followed by A

Sub-problems

Our sub-problems will be finding LCS's of prefixes to X and Y.

Let f[i,j] = length_of_LCS(X_i, Y_j)

- Case 2: X[i] != Y[j]

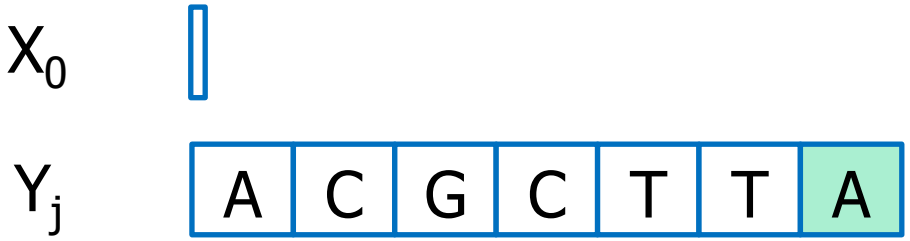


Notation: Denote this prefix ACGC by Y_4

- Then, f[i, j] = max(f[i-1, j], f[i,j-1]).
  - either LCS(X_i, Y_j) = LCS(X_{i-1}, Y_j) and T is not involved,
  - Or LCS(X_i, Y_j) = LCS(X_i, Y_{j-1}) and A is not involved.

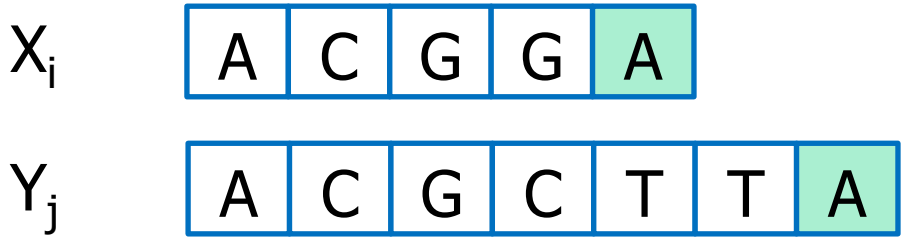
Recursive formulation

Case 0

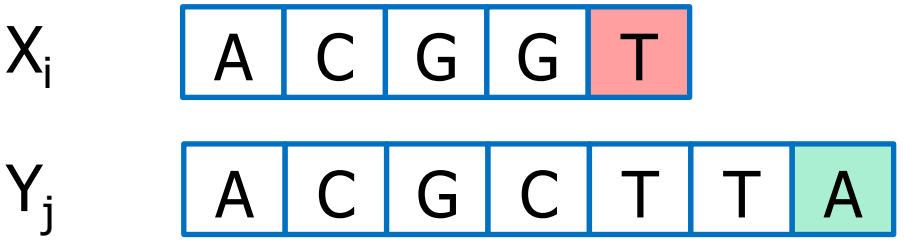


$$f[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + f[i-1, j-1] & \text{if } X[i]=Y[j] \\ \max(f[i-1, j], f[i,j-1]) & \text{if } X[i]!=Y[j] \end{cases}$$

Case 1



Case 2



LCS Example

$$f[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + f[i-1, j-1] & \text{if } X[i]=Y[j] \\ \max(f[i-1, j], f[i,j-1]) & \text{if } X[i]!=Y[j] \end{cases}$$

X

A	C	G	G	A

Y

A	C	T	G

		A	C	T	G
	0	0	0	0	0
A	0				
C	0				
G	0				
G	0				
A	0				

LCS Example

$$f[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + f[i-1, j-1] & \text{if } X[i]=Y[j] \\ \max(f[i-1, j], f[i,j-1]) & \text{if } X[i]!=Y[j] \end{cases}$$

X | A | C | G | G | A

Y | A | C | T | G

		A	C	T	G
	0	0	0	0	0
A	0	1	1	1	1
C	0	1	2	2	2
G	0	1	2	2	3
G	0	1	2	2	3
A	0	1	2	2	3

LCS Example

$$f[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + f[i-1, j-1] & \text{if } X[i]=Y[j] \\ \max(f[i-1, j], f[i,j-1]) & \text{if } X[i]!=Y[j] \end{cases}$$

X | A | C | G | G | A

Y | A | C | T | G

Once we've filled in, we can work backwards to get the LCS

		A	C	T	G
	0	0	0	0	0
A	0	1	1	1	1
C	0	1	2	2	2
G	0	1	2	2	3
G	0	1	2	2	3
A	0	1	2	2	3

LCS Example

$$f[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + f[i-1, j-1] & \text{if } X[i]=Y[j] \\ \max(f[i-1, j], f[i,j-1]) & \text{if } X[i]!=Y[j] \end{cases}$$

X | A | C | G | G | A

Y | A | C | T | G

That 3 comes from the 3 above it

		A	C	T	G
	0	0	0	0	0
A	0	1	1	1	1
C	0	1	2	2	2
G	0	1	2	2	3
G	0	1	2	2	3
A	0	1	2	2	3

LCS Example

$$f[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + f[i-1, j-1] & \text{if } X[i]=Y[j] \\ \max(f[i-1, j], f[i,j-1]) & \text{if } X[i]!=Y[j] \end{cases}$$

X | A | C | G | G | A

Y | A | C | T | G

That 3 comes from the 3 above it

		A	C	T	G
	0	0	0	0	0
A	0	1	1	1	1
C	0	1	2	2	2
G	0	1	2	2	3
G	0	1	2	2	3
A	0	1	2	2	3

LCS Example

$$f[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + f[i-1, j-1] & \text{if } X[i]=Y[j] \\ \max(f[i-1, j], f[i,j-1]) & \text{if } X[i]!=Y[j] \end{cases}$$

X | A | C | G | G | A

Y | A | C | T | G

A diagonal jump means that we found an element of the LCS!

		A	C	T	G
	0	0	0	0	0
A	0	1	1	1	1
C	0	1	2	2	2
G	0	1	2	2	3
G	0	1	2	2	3
A	0	1	2	2	3

LCS Example

$$f[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + f[i-1, j-1] & \text{if } X[i]=Y[j] \\ \max(f[i-1, j], f[i,j-1]) & \text{if } X[i]!=Y[j] \end{cases}$$

X	A C G G A	
Y	A C T G	

		A	C	T	G
	0	0	0	0	0
A	0	1	1	1	1
C	0	1	2	2	2
G	0	1	2	2	3
G	0	1	2	2	3
A	0	1	2	2	3

LCS Example

$$f[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + f[i-1, j-1] & \text{if } X[i]=Y[j] \\ \max(f[i-1, j], f[i,j-1]) & \text{if } X[i]!=Y[j] \end{cases}$$

X | A | C | G | G | A

Y | A | C | T | G

A diagonal jump means that we found an element of the LCS!

		A	C	T	G
	0	0	0	0	0
A	0	1	1	1	1
C	0	1	2	2	2
G	0	1	2	2	3
G	0	1	2	2	3
A	0	1	2	2	3

LCS Example

$$f[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + f[i-1, j-1] & \text{if } X[i]=Y[j] \\ \max(f[i-1, j], f[i,j-1]) & \text{if } X[i]!=Y[j] \end{cases}$$

X | A | C | G | G | A

Y | A | C | T | G

This is the LCS: | A | C | G |

		A	C	T	G
	0	0	0	0	0
A	0	1	1	1	1
C	0	1	2	2	2
G	0	1	2	2	3
G	0	1	2	2	3
A	0	1	2	2	3

Dynamic Programming Implementation

```
def LCS(X , Y):
    m = len(X)
    n = len(Y)
    f = [[None]*(n+1) for i in range(m+1)]

    for i in range(m+1):
        for j in range(n+1):
            if i == 0 or j == 0 :
                f[i][j] = 0
            elif X[i-1] == Y[j-1]:
                f[i][j] = f[i-1][j-1]+1
            else:
                f[i][j] = max(f[i-1][j] , f[i][j-1])

    return f[m][n]
```

Running time: O(nm)

```
X = "ACGGA"
Y = "ACTG"
print ("Length of LCS is ", LCS(X, Y) )
```

```
Length of LCS is  3
```

Exercise

- Write code to recover the actual LCS not just its length.
  - Time complexity?

Exercise 1: Climbing Stairs

- You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?
- Example:
  - Input: n = 3
  - Output: 3
  - Explanation: There are three ways to climb to the top.
    - 1. 1 step + 1 step + 1 step
    - 2. 1 step + 2 steps
    - 3. 2 steps + 1 step
- https://leetcode.com/problems/climbing-stairs/description/

Exercise 2: Longest Increasing Subsequence

- Subsequence:
  - [3,6,2,7] is a subsequence of [0,3,1,6,2,7]

- Use Dynamic Programming to find the length of the longest strictly increasing subsequence.
  - [0,1,0,3,2,3] : the longest increasing subsequence is [0,1,2,3], therefore the length is 4

- Find the actual longest increasing subsequence.

- https://leetcode.com/problems/longest-increasing-subsequence/description/